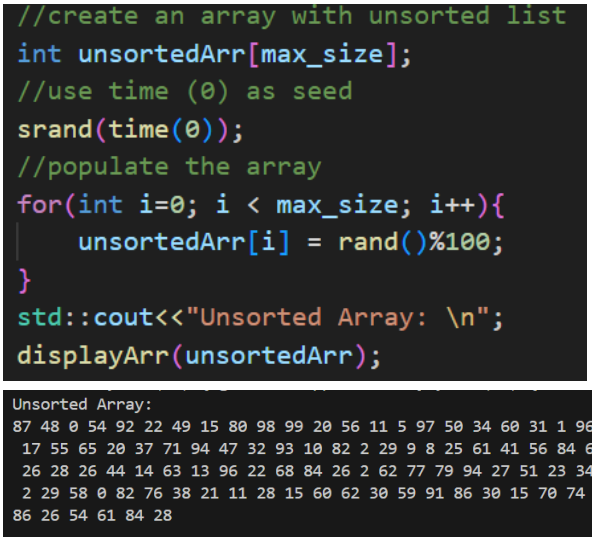
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 15, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	Macasaet, John David	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA NO. 7.1 **Sorting Algorithms Pt 2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	This is the array used for sorting algorithms

```
//create an array with unsorted list
int unsortedArr[max_size];
//use time (0) as seed
srand(time(0));
//populate the array
for(int i=0; i < max_size; i++){
    unsortedArr[i] = rand()%100;
}
std::cout<<"Unsorted Array: \n";
displayArr(unsortedArr);
```

```
Unsorted Array:
87 48 0 54 92 22 49 15 80 98 99 20 56 11 5 97 50 34 60 31 1 96 92 63
 17 55 65 20 37 71 94 47 32 93 10 82 2 29 9 8 25 61 41 56 84 65 45 8
 26 28 26 44 14 63 13 96 22 68 84 26 2 62 77 79 94 27 51 23 34 66 81
 2 29 58 0 82 76 38 21 11 28 15 60 62 30 59 91 86 30 15 70 74 59 88
86 26 54 61 84 28
```

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**sortingAlgo.h**

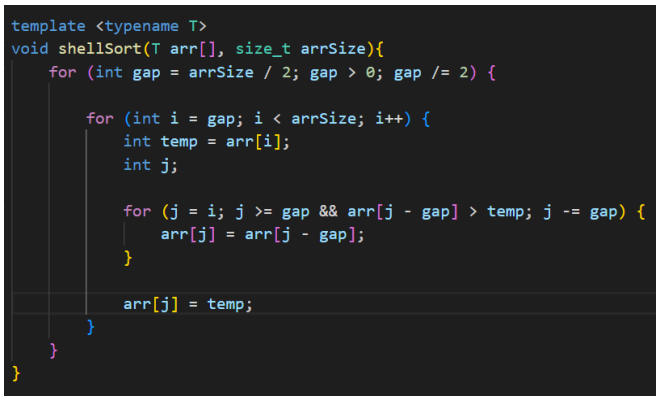
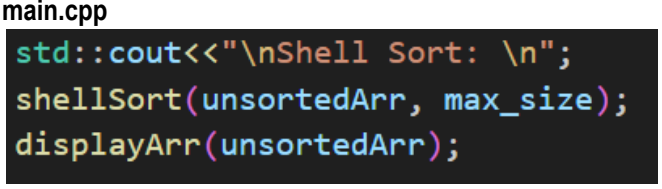
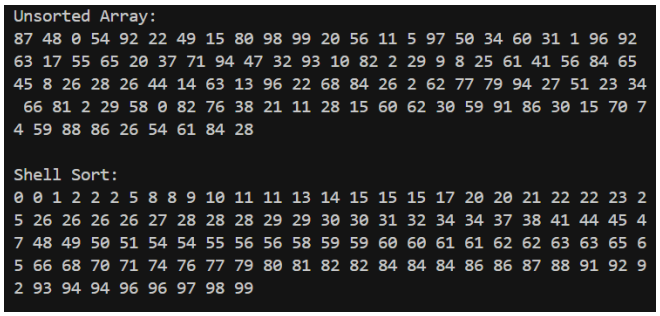
	 **main.cpp**  
Observations	**Shell Sort has a worst case time complexity of O(n^2) and best case of O(nlogn). This sorting algorithm is an improvement of insertion sort and introduces a gap and the goal is to keep sorting until the gap reduces.**

Table 8-2. Shell Sort Technique

Code + Console Screenshot	**sortingAlgo.h**

```cpp
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int n1 = middle - left + 1; //Calculates the number of items in the left side
    int n2 = right - middle; //Calculates the number of items in the right side
    //temporary arrays to store left and right side elements
    int L[n1];
    int R[n2];

    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[middle + 1 + j];
    }

    int i = 0;    // Initial index of left side array
    int j = 0;    // Initial index of right side array
    int k = left; // Initial index of merged

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements of L[] if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of R[] if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

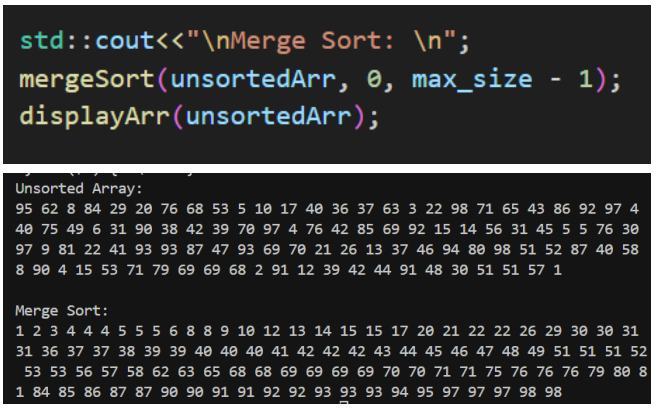
```cpp
template <typename T>
void mergeSort(T arr[], int left, int right) {
    // If left < right, then N > 1 (base condition to recurse)
    if (left < right) {
        // Find middle point
        int middle = (left + right) / 2;

        // Sort first half
        mergeSort(arr, left, middle);

        // Sort second half
        mergeSort(arr, middle + 1, right);

        // Merge sorted halves
        merge(arr, left, middle, right);
    }
}
```

main.cpp

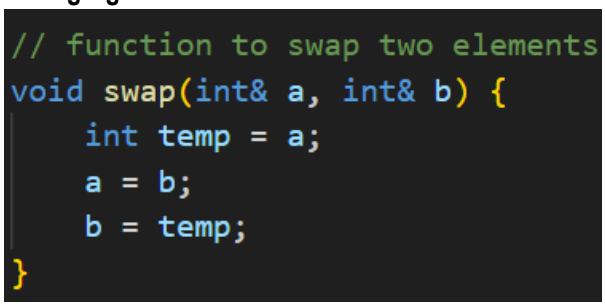
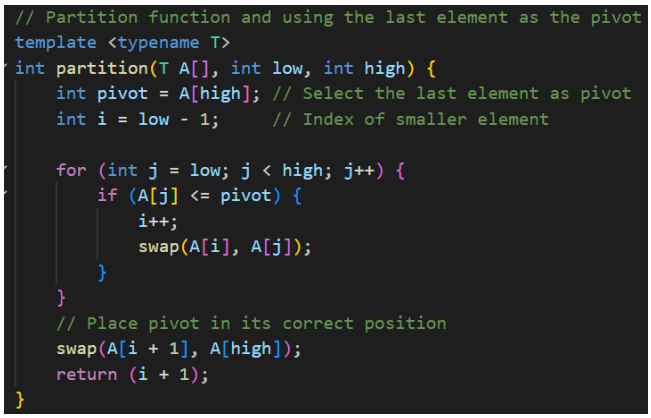
	
Observations	**The best and worst case time complexity is O(nlogn). This follows the divide and conquer rule where it divides the array and then merges them in the end but in a sorted order.**

```
std::cout<<"\nMerge Sort: \n";
mergeSort(unsortedArr, 0, max_size - 1);
displayArr(unsortedArr);
```

```
Unsorted Array:
95 62 8 84 29 20 76 68 53 5 10 17 40 36 37 63 3 22 98 71 65 43 86 92 97 4
40 75 49 6 31 90 38 42 39 70 97 4 76 42 85 69 92 15 14 56 31 45 5 5 76 30
97 9 81 22 41 93 93 87 47 93 69 70 21 26 13 37 46 94 80 98 51 52 87 40 58
8 90 4 15 53 71 79 69 69 68 2 91 12 39 42 44 91 48 30 51 51 57 1

Merge Sort:
1 2 3 4 4 4 5 5 5 6 8 8 9 10 12 13 14 15 15 17 20 21 22 22 26 29 30 30 31
31 36 37 37 38 39 39 40 40 40 41 42 42 42 43 44 45 46 47 48 49 51 51 51 52
 53 53 56 57 58 62 63 65 68 68 69 69 69 70 70 71 71 75 76 76 76 79 80 8
1 84 85 86 87 87 90 90 91 91 92 92 93 93 93 94 95 97 97 97 98 98
```

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	**sortingAlgo.h**  

```
// function to swap two elements
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

```
// Partition function and using the last element as the pivot
template <typename T>
int partition(T A[], int low, int high) {
    int pivot = A[high]; // Select the last element as pivot
    int i = low - 1;     // Index of smaller element

    for (int j = low; j < high; j++) {
        if (A[j] <= pivot) {
            i++;
            swap(A[i], A[j]);
        }
    }
    // Place pivot in its correct position
    swap(A[i + 1], A[high]);
    return (i + 1);
}
```

	main.cpp Unsorted Array: 31 29 80 7 51 43 30 47 78 30 21 73 68 89 4 61 42 3 24 61 64 13 17 6 21 36 33 11 95 20 66 8 67 3 20 75 60 31 71 47 47 51 42 88 97 94 55 95 32 9 17 14 23 34 16 59 93 78 80 81 55 92 92 83 9 18 80 20 74 96 1 15 91 99 28 51 34 92 21 29 83 20 89 57 27 6 63 59 3 11 83 24 75 5 40 71 16 7 9 3 95 Quick Sort: 1 3 3 3 3 4 5 6 6 7 8 9 9 11 11 13 14 15 16 16 17 17 18 20 20 20 20 21 21 21 23 24 24 27 28 29 29 30 30 31 31 32 33 34 34 36 40 42 42 43 47 47 47 51 51 51 55 55 57 59 59 60 61 61 63 64 66 67 68 71 71 73 74 75 7 5 78 78 79 80 80 80 81 83 83 83 88 89 89 91 92 92 92 93 94 95 95 95 96 97 99
Observations	**Quick sort has a worst case time complexity of O(n^2) and best case of O(nlogn). It introduces a pivot and then everything larger than the pivot goes to the right and the smaller ones go to the left.**

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1:

Using Quick Sort (Code)	Using Merge Sort (Code)

`std::cout<<"\nQuick Sort: \n";`
`quickSort(newArr, 0, newSize - 1);`
`displayArr(newArr);`	

```
//SUPPLEMENTARY PROB 1: USING MERGE SORT INSTEAD
//OF QUICK SORT
std::cout<<"\nMerge Sort with partition: \n";
partitionMergeSort(newArr, 0, newSize - 1);
displayArr(newArr);
```

OUTPUT (COMPARISON)

```
New Array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74

Quick Sort:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

Merge Sort with partition:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

Yes we can use different sorting algorithms instead of quick sort in sorting the left and right sub list form partition method. In this I used **merge sort**, I used it because it has a similar number of parameters in that it needs the array, and a starting index (low/left) and last index (high/right). As we can see from the output, they managed to both sort the array the same way.

PROBLEM 2:

Quick Sort is the fastest since it has a great time complexity with O(NlogN) but also that compared to merge sort which has the same time complexity, quick sort is more linear whereas merge sort needs to have extra temp arrays. That is the major difference between the two. Both merge sort and quick sort have a time complexity of O(NlogN) because both have for loop and/or while loops that make up N. And both require dividing by 2, in merge sort you divide by 2 when getting the middle. And in quick sort you divide the array into left and right using the partition method.

CONCLUSION:
In this activity, I implemented and analyzed different sorting algorithms like merge sort, quick sort, and shell sort. Most of this is analysis but that doesn't make it less challenging. By analysing them you can see which of the sorting algorithms are the most effective. And this is by knowing which one is the fastest in terms of performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI to give me clues on what to do if I'm stuck but I don't tell it to give me the code, I'm the one who thinks about how to create the code based on the clues AI gives me. I also did my own analysis of the code and just ask for recalling of concepts.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.